Unisoc Confidential For longcheer

Audio Smart PA 移植客制化指导手册

文档版本	V1.3
发布日期	2023-07-24

紫光展锐（上海）科技有限公司

版权所有 © 紫光展锐（上海）科技有限公司。保留一切权利。

本文件所含数据和信息都属于紫光展锐（上海）科技有限公司（以下简称紫光展锐）所有的机密信息，紫光展锐保留所有相关权利。本文件仅为信息参考之目的提供，不包含任何明示或默示的知识产权许可，也不表示有任何明示或默示的保证，包括但不限于满足任何特殊目的、不侵权或性能。当您接受这份文件时，即表示您同意本文件中内容和信息属于紫光展锐机密信息，且同意在未获得紫光展锐书面同意前，不使用或复制本文件的整体或部分，也不向任何其他方披露本文件内容。紫光展锐有权在未经事先通知的情况下，在任何时候对本文件做任何修改。紫光展锐对本文件所含数据和信息不做任何保证，在任何情况下，紫光展锐均不负责任何与本文件相关的直接或间接的、任何伤害或损失。

请参照交付物中说明文档对紫光展锐交付物进行使用，任何人对紫光展锐交付物的修改、定制化或违反说明文档的指引对紫光展锐交付物进行使用造成的任何损失由其自行承担。紫光展锐交付物中的性能指标、测试结果和参数等，均为在紫光展锐内部研发和测试系统中获得的，仅供参考，若任何人需要对交付物进行商用或量产，需要结合自身的软硬件测试环境进行全面的测试和调试。

商标声明

紫光展锐、UNISOC、、展讯、Spreadtrum、SPRD、锐迪科、RDA 及其他紫光展锐的商标均为紫光展锐（上海）科技有限公司及/或其子公司、关联公司所有。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

免责声明

本文档可能包含第三方内容，包括但不限于第三方信息、软件、组件、数据等。紫光展锐不控制且不对第三方内容承担任何责任，包括但不限于准确性、兼容性、可靠性、可用性、合法性、适当性、性能、不侵权、更新状态等，除非本文档另有明确说明。在本文档中提及或引用任何第三方内容不代表紫光展锐对第三方内容的认可、承诺或保证。

用户有义务结合自身情况，检查上述第三方内容的可用性。若需要第三方许可，应通过合法途径获取第三方许可，除非本文档另有明确说明。

Unisoc Confidential-For longcheer

紫光展锐（上海）科技有限公司

前 言

概述

本文以市面某厂商 NXP Smart PA 为例说明 Smart PA 移植的主要步骤，主要给 Audio 软件工程师和 Audio 软件相关工程师提供 Android 11、Android 12、Android 13、Android 14 平台 Smart PA 的集成做参考。

读者对象

本文档主要适用于 Audio 软件工程师。

缩略语

缩略语	英文全名	中文解释
API	Application Programming Interface	应用程序接口
DAI	Digital Audio Interface	数字音频接口
DAPM	Dynamic Audio Power Manager	Audio 动态电源管理
GKI	General Kernel Image	通用内核镜像

Unisoc Confidential For longcheer

符号约定

在本文中可能出现下列标志，它所代表的含义如下。

符号	说明
说明	用于突出重要或关键信息、补充信息和小窍门等。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害。
注意	用于突出容易出错的操作。 “注意”不是安全警示信息，不涉及人身、设备及环境伤害。
警告	用于可能无法恢复的失误操作。 “警告”不是危险警示信息，不涉及人身及环境伤害。

 版权所有 © 紫光展锐（上海）科技有限公司

变更信息

文档版本	发布日期	修改说明
V1.3	2023-07-24	新增适用于 Kernel5.15 的内容
V1.2	2023-01-09	• 新增适用于 Kernel5.4 的内容 • 优化文档内容
V1.1	2021-11-08	文档名称从《Android 11.0 Audio Smart PA 移植客制化指导手册》改为《Audio Smart PA 移植客制化指导手册》。
V1.0	2020-08-31	第一次正式发布。

关键字

Smart PA、IIS、Audio、HAL

Unisoc Confidential For longcheer

目 录

1 驱动修改 1
1.1 在 dts 中添加 I2C 节点配置 1
1.2 添加 dai link 2
1.3 配置 Kconfig/Makefile 2
1.4 修改驱动代码 4
1.5 切换 profile 的指令 5
2 Smart PA 校准及调试指令 6
2.1 调试时用到的节点 6
2.2 Smart PA 校准实现 6
3 Kcontrol 的使用及 HAL 层的修改 7
3.1 调整 tiny_hw.xml 7
3.2 配置 Board 宏 7
3.3 HAL 判断 Smart PA 存在的依据 8
3.4 Audio HAL IIS 配置 8
3.4.1 配置 Pin Mux 8
3.4.2 配置 IIS 模式 9
3.5 Smart PA 参数配置 11
3.5.1 参数文件路径 11
3.5.2 参数操作接口 11
3.6 Audio HAL Voice 相关配置 11
3.6.1 Voice 流程介绍 11
3.6.2 Voice 交互命令 14
3.6.3 Smart PA 相关接口介绍 15
3.6.4 Modem 音频参数修改 16
3.7 Media 相关配置 18
4 Audio Framework 相关修改 21
4.1 修改 AudioPolicy 21
4.2 修改 FM 22
4.2.1 修改 primary_audio_policy_configuration.xml 22
4.2.2 修改 audioflinger\Android.bp 24
4.2.3 修改 AudioFlinger.cpp 24
4.2.4 修改 Threads.cpp 27
4.2.5 修改 managerdefault\Android.bp 28
4.2.6 修改 AudioPolicyManager.cpp 28
5 常见问题 31

Unisoc Confidential For longcheer

5.1 声卡设备创建失败 31
5.2 播放音乐无声 31
6 附录 32
6.1 SprdFmSpk.h 32
6.2 SprdFmSpk.cpp 34

Unisoc Confidential For longcheer

图目录

图 3-1 Open 命令交互示意图.....12
图 3-2 Switch ctrl 命令交互示意图.....12
图 3-3 Device ctrl 命令交互示意图.....13
图 3-4 Set mode 命令交互示意图.....13
图 3-5 Set gain 命令交互示意图.....14
图 3-6 Close 命令交互示意图.....14
图 3-7 Smart amp 模式参数图.....16

Unisoc Confidential For longcheer

 版权所有 © 紫光展锐（上海）科技有限公司

表目录

表 3-1 Pin Mux 配置文件表 9
表 3-2 Voice 交互命令表 15
表 3-3 Smart PA 交互命令表 15
表 3-4 IIS 的配置参数 16

Unisoc Confidential For longcheer

 版权所有 © 紫光展锐（上海）科技有限公司

1 驱动修改

1.1 在 dts 中添加 I2C 节点配置

在 dts 中添加 I2C 节点配置，需要修改 dts 文件。该文件保存在“/arch/arm64/boot/dts/sprd/”。文件命名格式为<platform name>-<project name>.dts。

以 sp9863-1h10.dts 为例。添加 I2C 节点、设置 Smart PA 的 I2C 寻址地址及 GPIO reset 等信息相关代码如下所示。

```
&i2c2 {
    status = "okay";
    clock-frequency = <400000>;

    txxsmartpa:txx@36 {
        compatible = "nx,txxx";
        #sound-dai-cells = <1>;
        reg = <0x36>;
        reset-gpio= <&ap_gpio 60 0>;
    };
};
```

Unisoc Confidential For longcheer

- reset-gpio：按照实际硬件原理图填写，有时需要 irq 的 GPIO，配置类似。
- txxsmartpa:txx@36：thirdparty_smartpa 是此驱动节点的标号，后文中的 dai link 会用到。为了方便移植，建议所有第三方 Smart PA 节点标号都命名为 thirdparty_smartpa。
- reg：0x36 是 I2C 的 slave address，reg 属性需对应写上 0x36。硬件布线不同会影响 slave address，具体请参见芯片 spec 或联系展锐 FAE 获取支持。
- #sound-dai-cells = <1>：表示此驱动的声卡组成是有两个元素，默认为 1。如果填错会导致声卡解析失败。当驱动的 dai 只有一个时，可以填 0，也可以填 1。
- Android 11 及更低版本的 dts 文件路径
  /bsp/kernel/kernel4.14/arch/arm64/boot/dts/sprd/<platform name>-<project name>-xxxx.dts
- Android 12 的 dts 文件路径
  /bsp/kernel/kernel5.4/arch/arm64/boot/dts/sprd/<platform name>-<project name>-xxxx.dts
- Android 13 的 dts 文件路径
  /bsp/kernel5.4/kernel5.4/arch/arm64/boot/dts/sprd/<platform name>-<project name>-xxxx.dts
- Android 14 的 dts 文件路径
  /bsp/kernel5.15/kernel5.15/arch/arm64/boot/dts/sprd/<platform name>-<project name>-xxxx.dts

 版权所有 © 紫光展锐（上海）科技有限公司

1.2 添加 dai link

添加 dai link 相关代码如下：

```
&sound_sprd_ap_alliis {
	status = "okay";
	sprd-audio-card,dai-link@6 {
		plat {
			sound-dai = <&sprd_pcm_iis>;
		};
		cpu {
			sound-dai = <&i2s0>;
		};
		codec {
			sound-dai = <&tfasmartpa 0>;
		};
	};
```

pcm device 可在正常播放音乐后添加，相关代码如下所示。

```
sprd-audio-card,dai-link@7 {
		plat {
			sound-dai = <0>;
		};
		cpu {
			/* Voice through nxp smart pa(a dummy dai) */
			sound-dai = <&vaudio_iis 2>;
		};
		codec {
			sound-dai = <&tfasmartpa 1>;
		};
	};
};
```

Unisoc Confidential For longcheer

1.3 配置 Kconfig/Makefile

在“kernel4.14/sound/soc/codecs”目录下新建文件夹 txxx（第三方），将 NXP 的驱动代码复制到 txxx 中。

说明

- Android 12 的 codecs 路径为“/bsp/kernel/kernel5.4/sound/soc/codecs”。
- Android 13 的 codecs 路径为“/bsp/kernel5.4/kernel5.4/sound/soc/codecs”。

 版权所有 © 紫光展锐（上海）科技有限公司

- Android 14 的 codecs 路径为“/bsp/kernel5.15/kernel5.15/sound/soc/codecs”。

步骤 1 修改 sound/soc/codecs/Kconfig。

```
+config SND_SOC_TXXX
+    tristate "NXP TXXX smart amplifier"
+    depends on I2C
```

步骤 2 修改 sound/soc/codecs/Makefile。

```
+obj-$(CONFIG_SND_SOC_TXXX) += txxx/
```

步骤 3 在项目根目录下执行如下命令：

```
source build/envsetup.sh
lunch   your_project_name
kuconfig
```

kuconfig 执行成功，在 kernel 根目录下执行 git diff 命令，会看到 configs/sprd_xxxxxx_defconfig 文件中打开了宏 CONFIG_SND_SOC_TXXX。

```
+CONFIG_SND_SOC_TXXX=y
```

----结束

说明

- Android 11 及更低版本的 defconfig 路径为“bsp/kernel/kernel4.14/arch/arm64/configs”。
- Android 12 版本的 defconfig 路径为“bsp/kernel/kernel5.4/arch/arm64/configs”。
- Android 13 版本的 defconfig 路径为“bsp/kernel5.4/kernel5.4/arch/arm64/configs”。
- Android 14 版本的 defconfig 路径为“bsp/kernel5.15/kernel5.15/arch/arm64/configs”。

执行 kuconfig 后，如果其它与第三方驱动无关的宏也被打开，需要把这些宏移除。只需保留与第三方驱动有关的宏。

Unisoc Confidential For longcheer

Android 12 及更高版本的 Kernel

1. 以 ko 的方式编译和加载第三方驱动代码，并需要满足 GKI（General Kernel Image，通用内核镜像）的要求。Defconfig 中的修改如下：

```
+CONFIG_SND_SOC_TXXX=m
```

2. 在 ko 加载文件中，添加生成的第三方驱动 txxx.ko，并将相应的 ko 文件名放到 SPRD 的 codec ko（snd-soc-sprd-codec-sc2721.ko）后。
   - Android 12 路径

     device/sprd/mpool/module/bsp/mfeature/kernel/kernel5.4/msoc/ *platform name* /ko/modules.load

     device/sprd/mpool/module/bsp/mfeature/kernel/kernel5.4/msoc/ *platform name* /ko/km.mk
   - Android 13 路径

     device/sprd/mpool/module/others/bsp/mfeature/kernel/kernel5.4/msoc/ *platform name* /ko/modules.load

     device/sprd/mpool/module/others/bsp/mfeature/kernel/kernel5.4/msoc/ *platform name* /ko/km.mk
   - Android 14 路径

 版权所有 © 紫光展锐（上海）科技有限公司

device/sprd/vnd_mpool/module/vnd_others/bsp/mfeature/kernel/kernel5.15/msoc/ *platform name* /ko/modules.load

device/sprd/vnd_mpool/module/vnd_others/bsp/mfeature/kernel/kernel5.15/msoc/ *platform name* /ko/km.mk

1.4 修改驱动代码

在 **1.1** 在 dts 中添加 I2C 节点配置中，需要往 NXP 的驱动代码中添加解析代码，并做相应的调整。

如果前文的步骤中，在修改 dts 时添加了“sprd-audio-card,dai-link@7”节点，驱动中就需要添加下列修改内容（此内容可在能正常播放音乐后添加）。

代码路径为“sound/soc/codecs/txxx/txxx.c”。

添加代码如下：

```
@@ -1802,6 +1802,11 @@ static struct snd_soc_dapm_widget txxx_dapm_widgets_common[] = {
	/* Stream widgets */
	SND_SOC_DAPM_AIF_IN("AIF IN", "AIF Playback", 0, SND_SOC_NOPM, 0, 0),
	SND_SOC_DAPM_AIF_OUT("AIF OUT", "AIF Capture", 0, SND_SOC_NOPM, 0, 0),
+	/* Virt dai for voice */
+	SND_SOC_DAPM_AIF_IN("AIF IN Voice", "AIF Playback    Voice", 0,
+	                    SND_SOC_NOPM, 0, 0),
+	SND_SOC_DAPM_AIF_OUT("AIF OUT Voice", "AIF Capture Voice", 0,
+	                     SND_SOC_NOPM, 0, 0),

	SND_SOC_DAPM_OUTPUT("OUTL"),
	SND_SOC_DAPM_INPUT("AEC Loopback"),
@@ -3075,6 +3080,28 @@ static struct snd_soc_dai_driver txxx_dai[] = {
		.symmetric_samplebits = 1,
 #endif
	},
+	/* 1 - virt dai for voice call */
+	{
+		.name = "txxx-aif-voice",
+		.base = TXXX_TDM_CONFIG0 - 1,
+		.id = 1,
+		.playback = {
+			.stream_name = "AIF Playback Voice",
+			.channels_min = 1,
+			.channels_max = 4,
```

Unisoc Confidential For longcheer

```
+                .rates = TXXX_RATES,
+                .formats = TXXX_FORMATS,
+            },
+            .capture = {
+                .stream_name = "AIF Capture Voice",
+                .channels_min = 1,
+                .channels_max = 4,
+                .rates = TXXX_RATES,
+                .formats = TXXX_FORMATS,
+            },
+            .ops = &txxx_dai_ops,
+            .symmetric_rates = 1,
+    },
 };
```

1.5 切换 profile 的指令

完成 driver probe 后，会生成一些 Kcontrol。

以 Smart PA 挂在声卡 1 上为例。

Unisoc Confidential For longcheer

1. 可以执行下面的指令查看。

   ```
   tinymix -D 1
   ```

   看到 NXP 的下列 Kcontrol（可以在 cnt 文件中增加 Kcontrol 的数量）：

   ```
   19  ENUM   1   TXX Profile                    hp
   20  INT    1   TXX hp Playback Volume         0
   21  INT    1   TXX speech Playback Volume     0
   22  BOOL   1   TXX Stop                       Off
   ```

2. 用如下命令可查看 TXX Profile 可设置的值。

   ```
   tinymix -D 1 "TXX Profile"
   TXX Profile:  >hp  speech    standby
   ```

3. 用下列指令切换 profile。

   ```
   # tinymix "TXX Profile " "hp"
   # tinymix "TXX Profile " "speech"
   //非通话时设置为hp，通话时设置为speech
   ```

 版权所有 © 紫光展锐（上海）科技有限公司

2 Smart PA 校准及调试指令

2.1 调试时用到的节点

通常驱动中会新建一些用于 debug 的文件节点，一旦设备挂载成功后，就会生成文件节点。文件节点常保存在以下三种路径。

- /sys/bus/i2c/devices/i2c 序号/用于调试的各种文件节点名称
- /sys/kernel/debug/regmap/i2c 序号/用于调试的各种文件节点名称
- /sys/kernel/debug/i2c 设备名/用于调试的各种文件节点名称

例如：

```
/sys/kernel/debug/regmap/2-0036 # ls
Access
Name
Range
registers
/sys/kernel/debug/txxx-36 # ls
R
accounting
calibrate
dsp-state
regs
version
/sys/kernel/debug/txxx-36/regs # ls
00-STATUS          0A-I2S_SEL_REG
02-TEMPERATURE     04-I2SREG          09-SYS_CTRL
```

Unisoc Confidential For longcheer

2.2 Smart PA 校准实现

功能调试结束后，需要添加校准功能。校准功能需要集成到 MMI 测试中。在产线上可以测试 speaker 连接是否正常，speaker 选型是否正确，并计算和存储阻抗值。校准后的值会保存在 NV 中，Smart PA 在工作时会读取该值。

校准操作是必须的，因为部分厂家的 Smart PA 在没有校准的情况下，工作不稳定。不同厂家的校准方法各异，需要 Smart PA 厂家支持实现。

 版权所有 © 紫光展锐（上海）科技有限公司

3 Kcontrol 的使用及 HAL 层的修改

3.1 调整 tiny_hw.xml

将 Smart PA 驱动的 control 接口配置到“device/sprd/*platform name*/project name/module/audio/system/etc/audio_params/tiny_hw.xml”（适用 Android 11 和 Android 12）配置文件内。Audio HAL 会解析并根据场景（Audio 或 Voice）调用配置接口，并设置相应的 Smart PA 参数到 Smart PA 硬件。

```
<!-- scene: 0 - music; 1 - call. -->
<ext_codec name="Txx88">
    <config scene="0">
        <ctl name="TXX Profile" val="0"/>
    </config>
    <config scene="1">
        <ctl name="TXX Profile" val="1"/>
    </config>

    <!-- if the ext codec shared the i2s with bt sco -->
    <ap_i2s_shared shared="1"/>
</ext_codec>
```

Unisoc Confidential For longcheer

说明

- Android 13 配置文件路径“device/sprd/mpool/module/odm/audio/msoc/platform name /system/etc/audio_params/tiny_hw.xml”。
- Android 14 配置文件路径“device/sprd/vnd_mpool/module/odm/audio/msoc/platform name /system/etc/audio_params/tiny_hw.xml”。

3.2 配置 Board 宏

Board 宏的配置过程就是在对应文件中添加宏 AUDIO_SMART_PA_TYPE。

文件路径“/device/sprd/*chip*（例如 SC9863A）/*project*（例如 sp9863_1c10）/module/audio/md.mk”。

添加如下代码。

```
AUDIO_SMART_PA_TYPE := NXP
```

 版权所有 © 紫光展锐（上海）科技有限公司

3.3 HAL 判断 Smart PA 存在的依据

Audio_Hal 会根据以下代码中的文件节点判定是否有 Smart PA 设备。

```
#define EXT_SMARTPA_EXIST_FILE "/sys/kernel/ext_smartpa_exist"
static int get_ext_smartpa_exist(void)
{
    int fd;
    char exist = 1;
    char *file = EXT_SMARTPA_EXIST_FILE;
    if (!access(file , F_OK)) {
        fd = open(file , O_RDONLY);
        if (fd < 0) {
            ALOGE("%s, open '%s' failed!", __func__, file );
            return 1;
        }
        if (read(fd, &exist, sizeof(exist)) < 0) {
            ALOGE("%s, read '%s' failed!", __func__, file );
           fclose(fd);
            return 1;
        }
        exist = (exist != '0');
    }
    return exist;
}
```

Unisoc Confidential For longcheer

3.4 Audio HAL IIS 配置

Audio HAL IIS 配置包含 Pin Mux 配置和 IIS 模式配置。

3.4.1 配置 Pin Mux

Pin Mux 操作接口

- 函数所在的文件：audio_hw.c
- 函数名称：int i2s_pin_mux_sel(struct tiny_audio_device *adev, int type)
  其中，type 为需要切到 IIS 的目标，即 SoC 内部的模块。

 版权所有 © 紫光展锐（上海）科技有限公司

Pin Mux 配置文件路径

配置文件路径：/device/sprd/*chip*（例如 SC9863A）/*project*（例如 sp9863a1c10）/module/audio/system/etc/audio_params/audio_hw.xml（适用 Android 11 和 Android 12）

说明

- Android 13 的配置文件路径“device/sprd/mpool/module/odm/audio/msoc/platform name /system/etc/audio_params/tiny_hw.xml”。
- Android 14 配置文件路径“device/sprd/vnd_mpool/module/odm/audio/msoc/platform name /system/etc/audio_params/tiny_hw.xml”。

Pin Mux 配置文件说明

Pin Mux 配置文件如表 3-1 所示。

表3-1 Pin Mux 配置文件表

IIS 切换的目标	目标介绍	配置示例
CP_CSFB	IIS 切换到 modem DSP（通话场景）	<i2s_switch_btcall_inf3 ctl="SYS_IIS3" value="wcn_iis0"/> <i2s_switch_btcall_inf4 ctl="SYS_IIS4" value="tgdsp_iis0"/>
AP_TYPE	将 IIS 切到 AP 侧（普通播放等非通话场景）	<i2s_switch_ap_inf3 ctl="SYS_IIS3" value="wcn_iis0"/> <i2s_switch_ap_inf4 ctl="SYS_IIS4" value="ap_iis0"/> <i2s_switch_ap_inf0 ctl="SYS_IIS0" value="ap_iis1"/>
FM_IIS	IIS 切换切到 VBC（FM 播放场景）	<i2s_fm_bt_loop ctl="IIS_BT_FM_LOOP" value="1"/> <i2s_switch_fm_inf3 ctl="SYS_IIS3" value="wcn_iis0"/> <i2s_switch_fm_inf4 ctl="SYS_IIS4" value="vbc_iis1"/>
PUBCP	IIS 切换到 modem arm 的切换（HFP 场景会用到）	<i2s_switch_pubcp_inf3 ctl="SYS_IIS3" value="wcn_iis0"/> <i2s_switch_pubcp_inf4 ctl="SYS_IIS4" value="pubcp_iis0"/>
EXT_CODEC_AP	IIS 切换至 AP 侧	<i2s_switch_ext_codec_ap ctl="SYS_IIS1" value="ap_iis0"/>
EXT_CODEC_CP	IIS 切换至 modem DSP 侧	<i2s_switch_ext_codec_cp ctl="SYS_IIS1" value="tgdsp_iis1"/>

Unisoc Confidential for Longcheer

说明

- ctl：外部的 IIS 接口，例如 IIS0、IIS1、IIS2、IIS3。
- value：内部的模块，例如 vbc_iis1、ap_iis0、ap_iis1、tgdsp_iis0、tgdsp_iis1、pubcp_iis0、wcn_iis0。

3.4.2 配置 IIS 模式

- 操作接口函数名称：static int set_i2s_parameter(struct mixer *i2s_mixer,int i2s_type)

  i2s_type 需要传入参数“I2S_TYPE_EXT_CODEC”。
- SMART PA 的 IIS 配置。

  IIS 的配置在“vendor/sprd/modules/audio/normal/audio_hw.c”（适用 Android 11 版本）中，如下代码所示。

```
static const int ext_codec_configs[I2S_CONFIG_MAX] = {
```

 版权所有 © 紫光展锐（上海）科技有限公司

```
        48000, /* fs */
        0, /* hw_port */
        0xf11, /* slave_timeout */
        0, /* bus_type */
        1, /* byte_per_chan */
        0, /* mode(master/slave mode) */
        0, /* lsb */
        3, /* rtx_mode */
        0, /* lrck_inv(low_for_left) */
        1, /* sync_mode(lrck) */
        1, /* clk_inv */
        1, /* i2s_bus_mode */
        0, /* pcm_bus_mode */
        0, /* pcm_slot */
        0, /* pcm_cycle */
        16, /* tx_watermark */
        16, /* rx_watermark */
};
```

IIS 配置需要和 SMART PA 的 IIS 配置匹配。如果不匹配会导致播放无声。运行过程中，可通过 lookat 命令实时查看 IIS 寄存器是否已设置成功。IIS 的物理地址可通过查看 dts 文件里的 i2s 节点获得。

Unisoc Confidential - For Longcheer

例如：

kernel4.14/arch/arm64(arm)/boot/dts/sprd/*chip*.dtsi（例如 sharkl3.dtsi）（适用 Android 11 版本）

```
i2s0: i2s@70d00000 {
        compatible = "sprd,i2s";
        reg = <0 0x70d00000 0 0x100000>;
        sprd,dai_name = "i2s_bt_sco0";
        sprd,hw_port = <0>;
        sprd,syscon-ap-apb = <&ap_apb_regs>;
        #sound-dai-cells = <0>;
        status = "disable";
};
```

说明

- Android 12 的 dts 文件路径为“kernel/kernel5.4/arch/arm64(arm)/boot/dts/sprd/chip.dtsi”。
- Android 13 的 dts 文件路径为“kernel5.4/kernel5.4/arch/arm64(arm)/boot/dts/sprd/chip.dtsi”。
- Android 14 的 dts 文件路径为“kernel5.15/kernel5.15/arch/arm64(arm)/boot/dts/sprd/chip.dtsi”。
- Android 12、Android 13 和 Android14 的 audio_hw.c 文件路径为“vendor/sprd/modules/audio/vendor/normal/audio_hw.c”。

 版权所有 © 紫光展锐（上海）科技有限公司

3.5 Smart PA 参数配置

针对 NXP 的 Smart PA 有一个参数文件 cxx，在 Smart PA 开启时需要设置相应的场景的参数，使 Smart PA 工作正常。

3.5.1 参数文件路径

参数文件路径为“/device/sprd/*chip*（例如 SC9863A）/*project*（例如 sp9863_1c10）/module/audio/md.mk”。

```
PRODUCT_COPY_FILES += \
$(ROOTDIR)/system/etc/audio_params/txxx.cxx:$(TARGET_COPY_OUT_VENDOR) /firmware/tfa9896.cnt
```

3.5.2 参数操作接口

```
audio_hw.c:    static int set_ext_codec(struct tiny_audio_device *adev, int scene)
```

其中 scene 取值如下：

- 0：music
- 1：call

该接口在所有场景的 Smart PA 启动时，都会被调用，用于设置 Smart PA 的参数。

3.6 Audio HAL Voice 相关配置

Unisoc Confidential For longcheer

3.6.1 Voice 流程介绍

下面图 3-1 到图 3-6 是 AP 和 modem Audio 相关命令的交互示意图。交互过程中 Audio 的部分硬件由 AP 控制。

 版权所有 © 紫光展锐（上海）科技有限公司

图3-1 Open 命令交互示意图

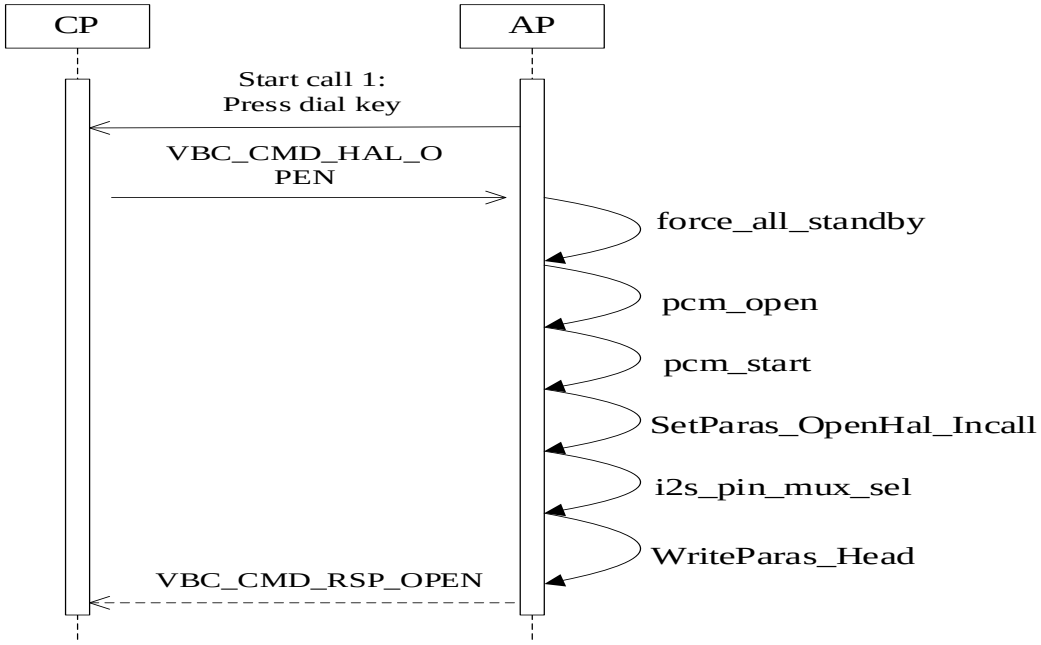
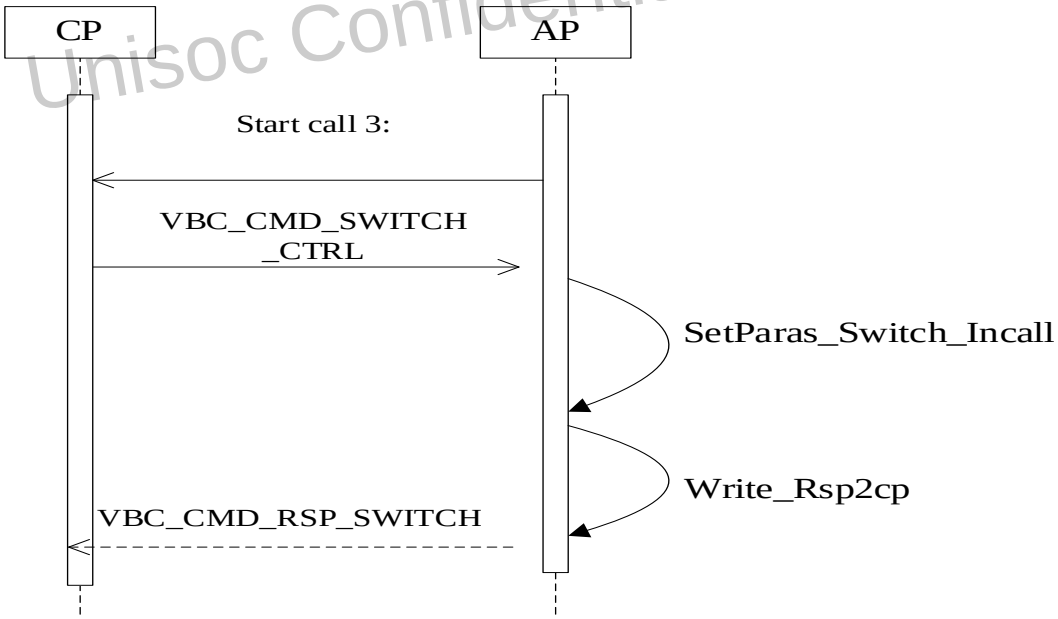


图3-2 Switch ctrl 命令交互示意图

Unisoc Confidential For longcheer



 版权所有 © 紫光展锐（上海）科技有限公司

图3-3 Device ctrl 命令交互示意图

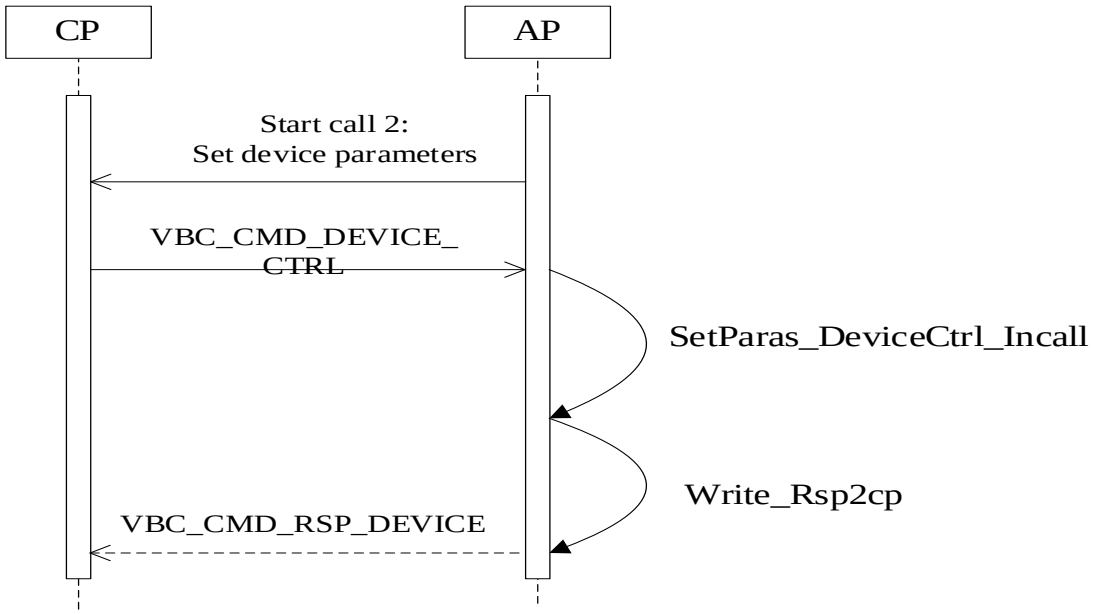
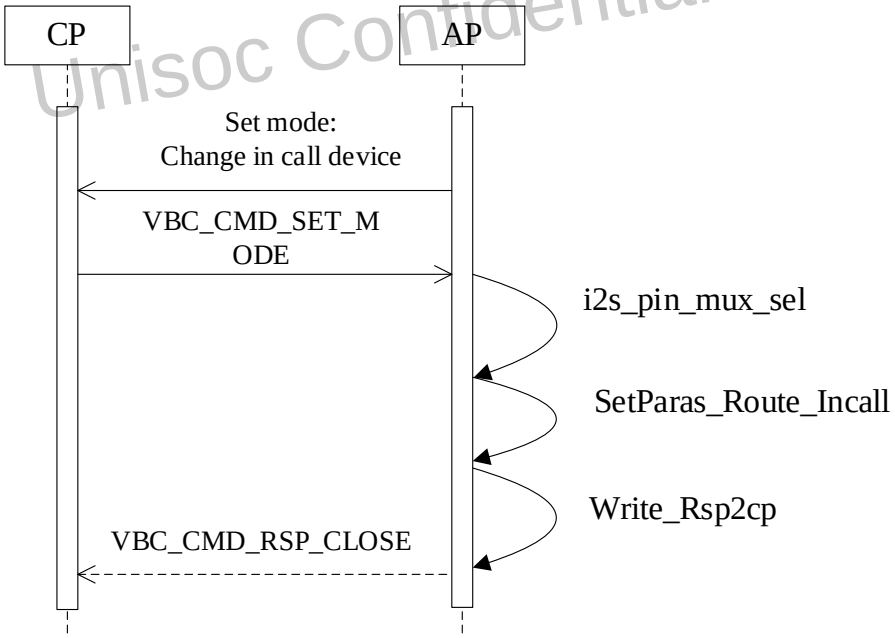


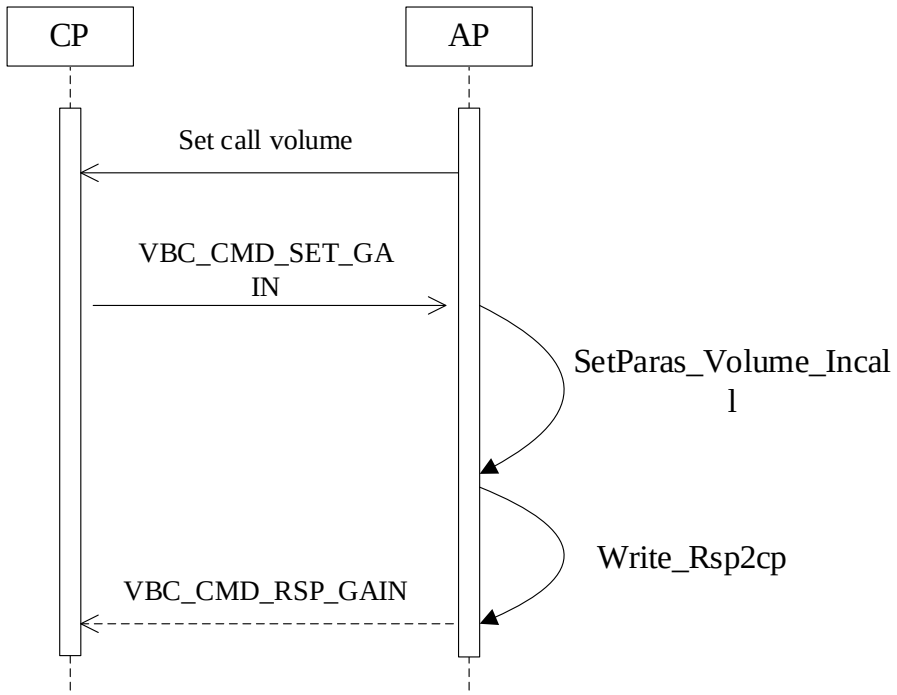
图3-4 Set mode 命令交互示意图

Unisoc Confidential For longcheer



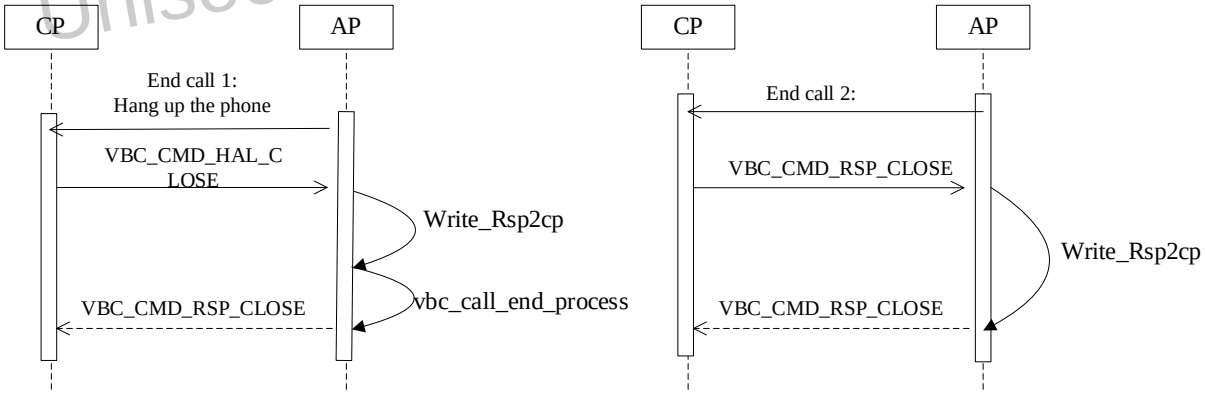
 版权所有 © 紫光展锐（上海）科技有限公司

图3-5 Set gain 命令交互示意图



Unisoc Confidential For longcheer

图3-6 Close 命令交互示意图



3.6.2 Voice 交互命令

Voice 交互命令情况如**表 3-2**。

 版权所有 © 紫光展锐（上海）科技有限公司

表3-2 Voice 交互命令表

序号	命令名称	说明
1	VBC_CMD_HAL_OPEN	建立 CP 和 AP 交互的第一个命令，关闭 AP 侧的非通话应用，开启 codec 的上下行通路。
2	VBC_CMD_DEVICE_CTRL	建立通话时设置 codec 相关参数的命令。
3	VBC_CMD_SET_MODE	模式设置命令，用于切换音频设备到相应的耳机、喇叭或听筒通路。
4	VBC_CMD_SET_GAIN	通话中调节音量时设置 codec 增益。
5	VBC_CMD_SWITCH_CTRL	切换 VBC 控制权，通话时将 VBC 控制权切到 DSP，结束通话时将 VBC 切到 AP。
6	VBC_CMD_SET_SAMPLERATE	网络模式等语音采样率变化时，更改 codec 输出采样率。
7	VBC_CMD_HAL_CLOSE	结束通话时关闭音频设备。

3.6.3 Smart PA 相关接口介绍

Smart PA 交互命令如**表 3-3** 所示。

表3-3 Smart PA 交互命令表

Unisoc Confidential For longcheer

序号	命令名称	说明

1	VBC_CMD_HAL_OPEN	通知 CP 存在外设 Smart PA。在 AP 回复 CP 的 VBC_CMD_RSP_OPEN 中，paras_size 参数设置 BIT10 为 1。例如：
		write_common_head.cmd_type = VBC_CMD_RSP_OPEN;
		write_common_head.paras_size = (1 << 10);

2	VBC_CMD_DEVICE_CTRL	设备控制。
3	VBC_CMD_SET_MODE	非 speaker 模式下，关闭 Smart PA 硬件 vbc_ctrl_nxp_tfa_close。
4	VBC_CMD_SET_GAIN	设置音量。
5	VBC_CMD_SWITCH_CTRL	切换控制，例如耳机、听筒、扬声器之间的切换。
6	VBC_CMD_SET_SAMPLERATE	开启 Smart PA 硬件 vbc_ctrl_nxp_tfa_open。 此时，DSP 已配置好 iis，iis 的 clock 已产生，可配置外部 Smart amp。
7	VBC_CMD_HAL_CLOSE	关闭 Smart PA 硬件 vbc_ctrl_nxp_tfa_close。

 版权所有 © 紫光展锐（上海）科技有限公司

开启 Smart PA 硬件

- 函数接口：void vbc_ctrl_nxp_tfa_open(struct tiny_audio_device *adev)
- 函数作用：
  - 将 iis 切换至对应的模块（CPU）
  - 打开外部设备 Smart PA 并设置对应场景的参数

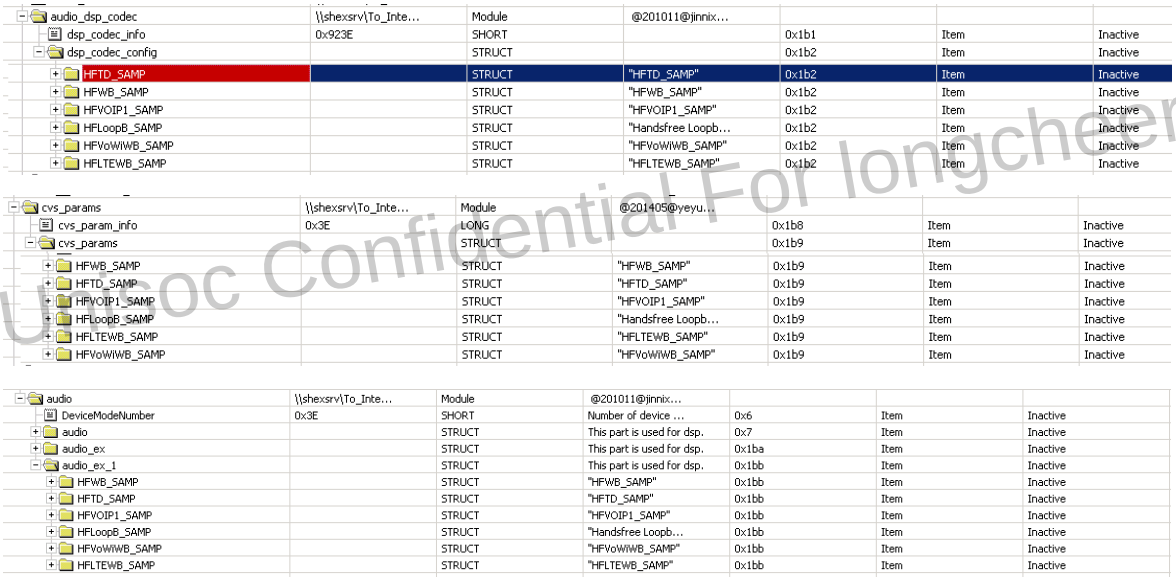
关闭 Smart PA 硬件

- 函数接口：void vbc_ctrl_nxp_tfa_close(struct tiny_audio_device *adev)
- 函数作用：
  - 将 IIS 还原至 AP
  - 关闭外部设备 Smart PA

3.6.4 Modem 音频参数修改

1. Modem 侧增加 Smart amp 的模式参数，具体请参见图 3-7。

图3-7 Smart amp 模式参数图



audio_dsp_codec	\\shexsrv\To_Inte...	Module	@201011@jinnix...			
dsp_codec_info	0x923E	SHORT		0x1b1	Item	Inactive
dsp_codec_config		STRUCT		0x1b2	Item	Inactive
HFTD_SAMP		STRUCT	"HFTD_SAMP"	0x1b2	Item	Inactive
HFWB_SAMP		STRUCT	"HFWB_SAMP"	0x1b2	Item	Inactive
HFVOIP1_SAMP		STRUCT	"HFVOIP1_SAMP"	0x1b2	Item	Inactive
HFLoopB_SAMP		STRUCT	"Handsfree Loopb...	0x1b2	Item	Inactive
HFVoWIWB_SAMP		STRUCT	"HFVoWIWB_SAMP"	0x1b2	Item	Inactive
HFLTEWB_SAMP		STRUCT	"HFLTEWB_SAMP"	0x1b2	Item	Inactive

cvs_params	\\shexsrv\To_Inte...	Module	@201405@yeyu...			
cvs_param_info	0x3E	LONG		0x1b8	Item	Inactive
cvs_params		STRUCT		0x1b9	Item	Inactive
HFWB_SAMP		STRUCT	"HFWB_SAMP"	0x1b9	Item	Inactive
HFTD_SAMP		STRUCT	"HFTD_SAMP"	0x1b9	Item	Inactive
HFVOIP1_SAMP		STRUCT	"HFVOIP1_SAMP"	0x1b9	Item	Inactive
HFLoopB_SAMP		STRUCT	"Handsfree Loopb...	0x1b9	Item	Inactive
HFLTEWB_SAMP		STRUCT	"HFLTEWB_SAMP"	0x1b9	Item	Inactive
HFVoWIWB_SAMP		STRUCT	"HFVoWIWB_SAMP"	0x1b9	Item	Inactive

audio	\\shexsrv\To_Inte...	Module	@201011@jinnix...			
DeviceModeNumber	0x3E	SHORT	Number of device ...	0x6	Item	Inactive
audio		STRUCT	This part is used for dsp.	0x7	Item	Inactive
audio_ex		STRUCT	This part is used for dsp.	0x1ba	Item	Inactive
audio_ex_1		STRUCT	This part is used for dsp.	0x1bb	Item	Inactive
HFWB_SAMP		STRUCT	"HFWB_SAMP"	0x1bb	Item	Inactive
HFTD_SAMP		STRUCT	"HFTD_SAMP"	0x1bb	Item	Inactive
HFVOIP1_SAMP		STRUCT	"HFVOIP1_SAMP"	0x1bb	Item	Inactive
HFLoopB_SAMP		STRUCT	"Handsfree Loopb...	0x1bb	Item	Inactive
HFVoWIWB_SAMP		STRUCT	"HFVoWIWB_SAMP"	0x1bb	Item	Inactive
HFLTEWB_SAMP		STRUCT	"HFLTEWB_SAMP"	0x1bb	Item	Inactive

Unisoc Confidential For longcheer

2. DSP 控制 IIS，IIS 的配置主要通过 NV 修改 Audio 下的 Extend2[118]部分内容，详见表 3-4。

表3-4 IIS 的配置参数

Extend2[118]		
Field Name	Bit	Description
reserved	[15:14]	reserved
pcm_slot	[13:11]	Pcm_slot “001”: slot 0 is used “010”: slot 1 is used

 版权所有 © 紫光展锐（上海）科技有限公司

Extend2[118]		
Field Name	Bit	Description
		“100”: slot 2 is used “011”: slot 0 and slot 1 are in use
trans_from_lsb	[10]	“0”: disable transmit data from LSB “1”: enable transmit data from LSB
bb_data_sel	[9]	BB used data select 0: low 16 bits 1: high 16 bits
Active_level_Left_chan	[8]	Active level of left channel 0: low for left channel 1: high for left channel
Bits_per_chan	[7:6]	Serial bits per channel 00: 0 bits 01: 16 bits 10: 32 bits 11: 无效
Sck_inverse	[5]	0: disable 1: enable
Operate mode	[4]	00: long frame mode in PCM mode MSB justified in IIS mode
Sync_mode	[3:2]	01: short frame mode in PCM mode lis-compatible in IIS mode
device_select	[1:0]	00:使用 BB 的 VB 01:使用 BB 的 PCM 10:使用 BB 的 IIS 11:无效

Unisoc Confidential For longcheer

3. DSP 内部通过分频系数分出 SCK，分频系数定义通过 NV 设置，如下所示。

   AudioStructure\extend2\extend2[117] = 0x7c

4. NV 里参数 Sample_rate 设置成 0x10。

   Sample_rate = 0x10

   - 0x10：16K
   - 0x8：8K

5. 修改 NV 参数。

   AudioStructure\vbc\reserved_for_new\reserved_for_new[1] = 0x1

   - 0：IIS0(VBC,BT)

 版权所有 © 紫光展锐（上海）科技有限公司

- 1：IIS1(OPEN IIS)

6. 修改 NV 参数 Echo Ref Signal Path Settings 项。

AudioStructure\vbc\reserved_for_new\reserved_for_new[3] = 0x2

- 0：DSP point 3 data 作为 Ref Signal 发送至 Arm®（PCM dsp 发送至音频硬件 VBC）。
- 1：DSP AuxMIC data 通过 VBC Loop 发送至 Arm®作为 Ref Signal。
- 2：DSP AuxMIC data 通过 codec 或 Smart_AMP Loop 至 VBC IIS TX 发送至 Arm®作为 Ref Signal。

3.7 Media 相关配置

playback

1. 操作接口：vendor/sprd/modules/audio/normal/nxppa/ Sprd_NxpTfa.c
   - 开启 Smart PA pcm 设备的函数接口

     nxp_pa_handle NxpTfa_Open(struct NxpTpa_Info *info, int need_src, int do_pcm_start)
   - 关闭 Smart PA pcm 设备的函数接口

     void NxpTfa_Close(nxp_pa_handle handle)
2. 在 speaker 播放时开启 Smart PA，非 speaker 情况下关闭 Smart PA。对应的文件和代码如下。
   - 文件：vendor/sprd/modules/audio/normal/nxppa/audio_hw.c
   - 代码

```
if (adev->smart_pa_exist) {
        if((!(out->is_bt_sco||out->is_voip || adev->call_start || (out->audio_app_type ==
            AUDIO_HW_APP_OFFLOAD)))
            && (out->devices & AUDIO_DEVICE_OUT_SPEAKER)) {
            if(!out->nxp_pa) {
                if(out->devices == AUDIO_DEVICE_OUT_SPEAKER) {
                    do_output_standby(out);
                }
                if (adev->ext_codec->i2s_shared)
                set_i2s_parameter(adev->i2s_mixer, I2S_TYPE_EXT_CODEC);
                i2s_pin_mux_sel_new(adev, EXT_CODEC, EXT_CODEC_AP);
                out->handle_pa = NxpTfa_Open(&NxpTfa9890_Info, 1, 0);
                if(out->handle_pa) {
                    out->nxp_pa = true;
                    set_ext_codec(adev, SCENE_MUSIC);
                    adev->active_output = out;
                    register_nxp_pcmdump(out->handle_pa,audio_pcm_dump);
                }
```

Unisoc Confidential For longcheer

 版权所有 © 紫光展锐（上海）科技有限公司

```
            }
        } else {
            if((out->nxp_pa == true) &&(!(out->devices & AUDIO_DEVICE_OUT_SPEAKER))) {
                out->nxp_pa = false;
                if(out->handle_pa) {
                    NxpTfa_Close(out->handle_pa);
                    out->handle_pa = NULL;
                }
            }
        }
    }
```

3. 写入数据至 Smart PA 的声卡设备。对应的文件和代码如下。
   - 文件：vendor/sprd/modules/audio/normal/nxppa/audio_hw.c
   - 代码

```
static ssize_t out_write(struct audio_stream_out *stream, const void* buffer,
        size_t bytes) {
 ……
    if(adev->smart_pa_exist && out->nxp_pa == true) {
        ret = NxpTfa_Write(out->handle_pa,buffer,bytes);
    }
……
}
```

Unisoc Confidential For longcheer

4. 在 speaker 播放关闭时关闭 Smart PA。对应的文件和代码如下。
   - 文件：vendor/sprd/modules/audio/normal/nxppa/audio_hw.c
   - 代码

```
static int do_output_standby(struct tiny_stream_out *out)
{
……
    if(adev->smart_pa_exist && out->nxp_pa) {
        out->nxp_pa = false;
        if(out->handle_pa) {
            NxpTfa_Close(out->handle_pa);
            out->handle_pa = NULL;
        }
    }
……
}
```

 版权所有 © 紫光展锐（上海）科技有限公司

说明

- Android 12、Android 13 和 Android 14 的 Sprd_NxpTfa.c 文件路径为“vendor/sprd/modules/audio/vendor/normal/nxppa/ Sprd_NxpTfa.c”。
- Android 12、Android 13 和 Android 14 的 audio_hw.c 文件路径为“vendor/sprd/modules/audio/vendor/normal/nxppa/audio_hw.c”。

Unisoc Confidential For longcheer

4 Audio Framework 相关修改

4.1 修改 AudioPolicy

1. 开启 Smart PA 后，speaker 模式播放的情况下只能有一个 IIS 输出。mp3 offload 无法支持，需要修改 audiopolicy 配置文件来关闭。对应的文件和代码如下。
   - 文件：/device/sprd/*chip*（例如 SC9863A）/*project*（例如 sp98631c10）/module/audio/system/etc/audio_policy_config/primary_audio_policy_configuration.xml（适用 Android 11 和 Android 12）。
   - 代码

```
    <routes>
        <route type="mix" sink="Earpiece"
                sources="primary output,compressed_offload,BT SCO Headset Mic"/>
        <route type="mix" sink="Speaker"
-               sources="primary output,compressed_offload,BT SCO Headset Mic,Telephony Rx
"/>
+               sources="primary output,BT SCO Headset Mic,Telephony Rx"/>
        <route type="mix" sink="Wired Headset"
            sources="primary output,compressed_offload,BT SCO Headset Mic,Telephony Rx"/>
```

Unisoc Confidential For longcheer

2. FM 增加一个录音 source。FM 播放时，CPU 先通过 FM source Mic 将 FM PCM 录上，再通过 audiotrack 播放。对应代码如下。

```
    <attachedDevices>
        <item>Speaker</item>
        <item>Earpiece</item>
        <item>Built-In Mic</item>
        <item>Telephony Rx</item>
-       <item>Telephony Tx</item>
+       <item>FM source Mic</item>
    </attachedDevices>
    <defaultOutputDevice>Speaker</defaultOutputDevice>
    <mixPorts>
        <mixPort name="primary output" role="source" flags="AUDIO_OUTPUT_FLAG_PRIMARY">
            <profile name="" format="AUDIO_FORMAT_PCM_16_BIT"
                    samplingRates="44100"
channelMasks="AUDIO_CHANNEL_OUT_STEREO"/>
```

 版权所有 © 紫光展锐（上海）科技有限公司

```
        </mixPort>
  +       <mixPorts>
  +       <mixPort name="fm_source_in" role="sink">
  +           <profile name="" format="AUDIO_FORMAT_PCM_16_BIT"
  +                   samplingRates="44100"
+                   channelMasks="AUDIO_CHANNEL_IN_STEREO"/>
+       </mixPort>

  <route type="mix" sink="primary input"
  -           sources="Built-In Mic,Wired Headset Mic,BT SCO Headset Mic,Telephony Rx"/>
  +           sources="Built-In Mic,Wired Headset Mic,BT SCO Headset Mic,Telephony Rx, FM
source Mic "/>
```

说明

- Android 13 的配置文件路径为“device/sprd/mpool/module/odm/audio/msoc//platform name /system/etc/audio_policy_config/primary_audio_policy_configuration.xml”。
- Android 14 的配置文件路径为“device/sprd/vnd_mpool/module/odm/audio/msoc//platform name /system/etc/audio_policy_config/primary_audio_policy_configuration.xml”。

Unisoc Confidential For longcheer

4.2 修改 FM

FM 由于没有硬件通路直接通往 IIS，所以需要 CPU 通过 VBC 将 FM 数据由录音接口读取，再和音乐一样的方式播放。

在 audioflinger 里上层将 FM 切换至 speaker 时，开启 FM PCM 录音并通过 Audiotrack 播放。切换至非 speaker 播放时，关闭 FM PCM 录音和播放的通路，切至 VBC 硬件通路播放。修改点下面小节进行具体描述。

4.2.1 修改 primary_audio_policy_configuration.xml

- 文件：/module/audio/system/etc/audio_policy_config/primary_audio_policy_configuration.xml
- 代码
  - 修改点 1

```
<mixPort name="primary output" role="source" flags="AUDIO_OUTPUT_FLAG_PRIMARY">
        <profile name="" format="AUDIO_FORMAT_PCM_16_BIT"
            samplingRates="44100"
channelMasks="AUDIO_CHANNEL_OUT_STEREO"/>
</mixPort>
-<mixPort name="compressed_offload" role="source"
flags="AUDIO_OUTPUT_FLAG_DIRECT|AUDIO_OUTPUT_FLAG_COMPRESS_OFFLOAD|AU
DIO_OUTPUT_FLAG_NON_BLOCKING">
```

 版权所有 © 紫光展锐（上海）科技有限公司

```
-       <profile name="" format="AUDIO_FORMAT_MP3"
-                samplingRates="8000,11025,12000,16000,22050,24000,32000,44100,48000"
-
channelMasks="AUDIO_CHANNEL_OUT_STEREO,AUDIO_CHANNEL_OUT_MONO"/>
-</mixPort>
<mixPort name="voice_tx" role="source">
        <profile name="" format="AUDIO_FORMAT_PCM_16_BIT"
          samplingRates="8000,16000"
   channelMasks="AUDIO_CHANNEL_OUT_MONO"/>
</mixPort>
```

- 修改点 2

```
<routes>
        <route type="mix" sink="Earpiece"
-                sources="primary output,compressed_offload,BT SCO Headset Mic"/>
+                sources="primary output,BT SCO Headset Mic"/>
        <route type="mix" sink="Speaker"
                sources="primary output,BT SCO Headset Mic,Telephony Rx"/>
        <route type="mix" sink="Wired Headset"
-                sources="primary output,compressed_offload,BT SCO Headset Mic,Telephony Rx"/>
+                sources="primary output,BT SCO Headset Mic,Telephony Rx"/>
        <route type="mix" sink="Wired Headphones"
-                sources="primary output,compressed_offload,BT SCO Headset Mic,Telephony Rx"/>
+                sources="primary output,BT SCO Headset Mic,Telephony Rx"/>
        <route type="mix" sink="BT SCO"
                sources="primary output,Telephony Rx"/>
        <route type="mix" sink="BT SCO Headset"
                sources="primary output,Telephony Rx"/>
         <route type="mix" sink="BT SCO Car Kit"
                sources="primary output,Telephony Rx"/>
         <route type="mix" sink="primary input"
                sources="Built-In Mic,Built-In Back Mic,Wired Headset Mic,BT SCO Headset Mic,FM
source Mic"/>
         <route type="mix" sink="Telephony Tx"
                sources="Built-In Mic,Built-In Back Mic,Wired Headset Mic,BT SCO Headset
Mic,voice_tx"/>
         <route type="mix" sink="voice_rx"
                sources="Telephony Rx"/>
         <route type="mix" sink="fm_source_in"
```

Unisoc Confidential For longcheer

```
                    sources="FM source Mic"/>
</routes>
```

4.2.2 修改 audioflinger\Android.bp

修改的文件和代码如下。

- 文件：frameworks/av/services/audioflinger/Android.bp
- 代码修改点

```
    "SpdifStreamOut.cpp",
+ "SprdFmSpk.cpp",
     "StateQueue.cpp",
     "Threads.cpp",
     "Tracks.cpp",

cflags: [
          "-DSTATE_QUEUE_INSTANTIATIONS=\"StateQueueInstantiations.cpp\"",
           "-fvisibility=hidden",
           "-Werror",
           "-Wall",
+         "-DNXP_SMART_PA"
     ],
```

Unisoc Confidential For longcheer

4.2.3 修改 AudioFlinger.cpp

修改的文件和代码如下。

- 文件：frameworks/av/services/audioflinger/AudioFlinger.cpp
- 代码
  - 修改点 1

```
#include "TypedLogger.h"
+ #ifdef NXP_SMART_PA
+ #include <SprdFmSpk.h>
+ #endif
#define AUDIO_HARDWARE_MODULE_ID_BLUETOOTH "bluetooth"
```

  - 修改点 2

```
+#ifdef NXP_SMART_PA
+sp<sprdFmSpk> g_spk_fm=NULL;
+#define EXT_SMARTPA_EXIST_FILE "/sys/kernel/ext_smartpa_exist"
+static int get_ext_smartpa_exist(void)
+{
```

 版权所有 © 紫光展锐（上海）科技有限公司

```
+       int fd;
+       char exist = 1;
+       char *file = (char *)EXT_SMARTPA_EXIST_FILE;

+       if (!access(file , F_OK)) {
+               fd = open(file , O_RDONLY);
+               if (fd < 0) {
+                       ALOGE("%s, open '%s' failed!", __func__, file );
+                       return 1;
+               }
+             if (read(fd, &exist, sizeof(exist)) < 0) {
+                       ALOGE("%s, read '%s' failed!", __func__, file );
+                       return 1;
+               }

+               exist = (exist != '0');
+       }

+       ALOGI("External smartpa exist: %d", exist);

+       return exist;
+}
+#endif

        /* get property and set mSYSTEM_READY */
        const bool systemreadyStatus = property_get_bool("af.media.systemready.state", false);
        if (systemreadyStatus) {
            mSystemReady = true;
        }

+ #ifdef NXP_SMART_PA
+         if(get_ext_smartpa_exist()) {
+                 g_spk_fm = new    sprdFmSpk();
+                 ALOGI("smart pa is exist");
+         }
+#endif
```

Unisoc Confidential For longcheer

```
    const bool doLog = property_get_bool("ro.test_harness", false);
    if (doLog) {
        mLogMemoryDealer = new MemoryDealer(kLogMemorySize, "LogWriters",
                MemoryHeapBase::READ_ONLY);
        (void) pthread_once(&sMediaLogOnce, sMediaLogInit);
    }
```

- 修改点 3

```
//添加宏NXP_SMART_PA控制的部分。
        // disable AEC and NS if the device is a BT SCO headset supporting those pre processings
        AudioParameter param = AudioParameter(filteredKeyValuePairs);
        String8 value;
+#ifdef NXP_SMART_PA
+         if(param.get(String8("handleFm"),value) == NO_ERROR){
+             bool fmIsOn = (value == "1");
+             if(g_spk_fm != NULL) {
+                 if(fmIsOn) {
+                     g_spk_fm->fm_on();
+                 }
+                 else {
+                     g_spk_fm->fm_off();
+                 }
+             }
+         }
+#endif
        if (param.get(String8(AudioParameter::keyBtNrec), value) == NO_ERROR) {
            bool btNrecIsOff = (value == AudioParameter::valueOff);
            if (mBtNrecIsOff.exchange(btNrecIsOff) != btNrecIsOff) {
                for (size_t i = 0; i < mRecordThreads.size(); i+) {
                    mRecordThreads.valueAt(i)->checkBtNrec();
                }
            }
        }



    if (thread != 0) {
```

- 修改点 4

```
//添加宏NXP_SMART_PA控制的部分。
```

Unisoc Confidential For longcheer

```
+#ifdef NXP_SMART_PA
+            AudioParameter param = AudioParameter(filteredKeyValuePairs);
+            int value;
+            if(param.getInt(String8(AUDIO_PARAMETER_STREAM_ROUTING),value) ==
NO_ERROR){
+                if(g_spk_fm != NULL) {
+                    if((int)value &AUDIO_DEVICE_OUT_SPEAKER) {
+                        g_spk_fm->start();
+                    }
+                    else {
+                        g_spk_fm->stop();
+                    }
+                }
+            }
+#endif
            status_t result = thread->setParameters(filteredKeyValuePairs);
            forwardParametersToDownstreamPatches_l(thread->id(), filteredKeyValuePairs);
            return result;
        }
```

Unisoc Confidential For longcheer

4.2.4 修改 Threads.cpp

修改的文件和代码如下。

- 文件：frameworks/av/services/audioflinger/Threads.cpp
- 代码修改点

```
+#ifdef NXP_SMART_PA
+#include <SprdFmSpk.h>
+#endif


+#ifdef NXP_SMART_PA
+extern sp<sprdFmSpk> g_spk_fm;
+#endif
status_t AudioFlinger::PlaybackThread::createAudioPatch_l(const struct audio_patch *patch,


....
    if (mOutput->audioHwDev->supportsAudioPatches()) {
        sp<DeviceHalInterface> hwDevice = mOutput->audioHwDev->hwDevice();
```

 版权所有 © 紫光展锐（上海）科技有限公司

```
        status = hwDevice->createAudioPatch(patch->num_sources,
                                            patch->sources,
                                            patch->num_sinks,
                                            patch->sinks,
                                            handle);
+#ifdef NXP_SMART_PA
+           if(g_spk_fm != NULL) {
+               if((int)type &AUDIO_DEVICE_OUT_SPEAKER) {
+                   g_spk_fm->start();
+               }
+               else {
+                   g_spk_fm->stop();
+               }
+           }
+#endif
    } else {
```

4.2.5 修改 managerdefault\Android.bp

修改的文件和代码如下。

- 文件：frameworks/av/services/audiopolicy/managerdefault/Android.bp
- 代码修改点

```
    cflags: [
        "-Wall",
        "-Werror",
        "-DSPRD_CUSTOM_AUDIO_POLICY",
+         "-DNXP_SMART_PA",
    ],
```

Unisoc Confidential For longcheer

4.2.6 修改 AudioPolicyManager.cpp

修改的文件和代码如下。

- 文件：frameworks/av/services/audiopolicy/managerdefault/AudioPolicyManager.cpp
- 代码
  - 修改点 1

```
//添加宏NXP_SMART_PA控制的部分。
uint32_t AudioPolicyManager::nextAudioPortGeneration()
{
    return mAudioPortGeneration+;
```

 版权所有 © 紫光展锐（上海）科技有限公司

```
}

+#ifdef NXP_SMART_PA
+#define EXT_SMARTPA_EXIST_FILE "/sys/kernel/ext_smartpa_exist"
+static int get_ext_smartpa_exist(void)
+{
+       int fd;
+       char exist = 1;
+       char *file = (char *)EXT_SMARTPA_EXIST_FILE;

+       if (!access(file , F_OK)) {
+               fd = open(file , O_RDONLY);
+               if (fd < 0) {
+                       ALOGE("%s, open '%s' failed!", __func__, file );
+                       return 1;
+               }
+               if (read(fd, &exist, sizeof(exist)) < 0) {
+                       ALOGE("%s, read '%s' failed!", __func__, file );
+                       return 1;
+               }

+               exist = (exist != '0');
+       }

+       ALOGI("External smartpa exist: %d", exist);

+       return exist;
+}
+#endif

static status_t deserializeAudioPolicyXmlConfig(AudioPolicyConfig &config) {
```

– 新增的代码文件

下面两个文件的具体源码请参见章节 **6** 附录。

- /framework/av/services/audioflinger/SprdFmSpk.cpp
- /framework/av/services/audioflinger/SprdFmSpk.h

```
status_t AudioFlinger::PlaybackThread::createAudioPatch_l(const struct audio_patch *patch,
audio_patch_handle_t *handle)
{
```

Unisoc Confidential For longcheer

```
……
        param.addInt(String8(AudioParameter::keyRouting), (int)type);
#ifdef NXP_SMART_PA
        if(g_spk_fm != NULL) {
            if((int)type &AUDIO_DEVICE_OUT_SPEAKER) {
                g_spk_fm->start();
            }
            else {
                g_spk_fm->stop();
            }
        }
#endif
    ……
}
```

Unisoc Confidential For longcheer

5 常见问题

5.1 声卡设备创建失败

【原因分析】

tfa98xx_i2c_probe 执行成功了，但是 tfa98xx_probe 没有执行。

【解决方法】

可能是 dai link 未修改或错误修改导致，需要调整 dts 中关于 dai link 的配置。

5.2 播放音乐无声

【原因分析】

可能存在多种原因导致无声，例如 IIS 信号异常、tiny_hw.xml 配置错误等。

Unisoc Confidential For longcheer

【解决方法】

播放音乐时，用示波器检查 IIS 的 LR、DATA、clock 上是否有信号。

- 如果无信号或信号异常，先调试 IIS。
- 如果 IIS 信号正常，检查 tiny_hw.xml 配置等是否正常，以及驱动中是否成功注册 Kcontrol。

 版权所有 © 紫光展锐（上海）科技有限公司

6 附录

6.1 SprdFmSpk.h

```
#ifndef __SPRD_SPK_FM_INTERFACE__
#define    __SPRD_SPK_FM_INTERFACE__
#include <errno.h>
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <linux/types.h>
#include <media/AudioRecord.h>
#include <media/AudioTrack.h>
#include <utils/RefBase.h>
namespace android {
class     sprdFmSpk:public Thread {

public:
      sprdFmSpk();
      ~sprdFmSpk();
      status_t    start();
      status_t    stop();
      status_t fm_on();
      status_t fm_off();
        void          exit();


                    // Thread virtuals
      virtual status_t       readyToRun() { return NO_ERROR; }
      virtual void              onFirstRef();
     virtual        bool              threadLoop();
        // base for record and playback
      enum fm_command {
            CMD_START,
            CMD_STOP
```

Unisoc Confidential For longcheer

```
    };
    enum fm_state{
        STATE_IDLE,
        STATE_START,
        STATE_STOP,
        STATE_EXIT
    };

class Command: public RefBase {
public:
        Command(int type,bool wait = false) :
        mCmd(type), mStatus(NO_ERROR), mWaitStatus(wait){}
    virtual ~Command() {}

    const int mCmd; // event type e.g. CFG_EVENT_IO
    Mutex mLock;  // mutex associated with mCond
    Condition mCond; // condition for status return
    status_t mStatus; // status communicated to sender
    bool mWaitStatus; // true if sender is waiting for status
};
private:
    void processCmd_l();
    status_t sendCommand(fm_command cmd,bool wait);
    status_t sendCommand_l(sp<Command>& event);
    status_t cmd_start_l();
    void cmd_stop_l();
    status_t playback_start_l();
    void playback_stop_l();
    status_t record_start_l();
    void record_stop_l();
    status_t    start_l(bool sync);
        status_t    stop_l(bool sync);

private:
    Mutex mCmdLock;
    Vector< sp<Command> >        mCommands;
    fm_state mState;
```

Unisoc Confidential For longcheer

```
    fm_state mInterfaceState;
    Mutex mLock;
    sp<AudioRecord> mRecord;
    sp<AudioTrack> mAudioTrack;
    int8_t * mBuffer;
    uint32_t mBufferSize;
    Condition    mWaitWorkCV;
    bool mFmON;
};
}
#endif
```

6.2 SprdFmSpk.cpp

```
#define LOG_TAG "sprdfmspk"
#include <string.h>
#include <utils/Log.h>
#include <system/audio.h>
#include <system/audio_policy.h>
#include <errno.h>
#include <pthread.h>
#include <stdio.h>
#include <utils/Log.h>
#include <stdlib.h>
#include <linux/types.h>
#include "SprdFmSpk.h"
namespace android {
bool    sprdFmSpk::threadLoop()
{
    char *buffer = NULL;
    int status;
    int is_block = 1;
    int bytes_read = 0;
    ALOGE("SprdFm_Spk_threadentry Loop start");
    while(!exitPending())
    {
        mCmdLock.lock();
```

Unisoc Confidential For longcheer

```
        if(mCommands.isEmpty()) {
            if((mRecord == NULL ) || (mAudioTrack == NULL)) {
                ALOGI("wait for work in");
                 mWaitWorkCV.wait(mCmdLock);
                 ALOGI("wait for work out");
            }
        }
        else {
            processCmd_l();
        }
        mCmdLock.unlock();
        if((mState== STATE_START)&& (mRecord != NULL) &&(mAudioTrack != NULL)&&mBuffer)
        {
            bytes_read = mRecord->read(mBuffer, mBufferSize, is_block);
            if(bytes_read == 0)
            {
                ALOGE("Sprdfm:Read Pcm_Tfa_Fm Error! ret = %d", bytes_read);
                usleep(20000);
                continue;
            }
            status = mAudioTrack->write( mBuffer, (size_t)bytes_read,    (bool)1);
            if(status < 0)
            {
                ALOGE("Sprdfm:FmTrack Write ERR status = %d", status);
            }
        }
        else
        {
            usleep(10000);
        }
    }
    ALOGE("SprdFm:RW thread exit!");
    if (buffer != NULL)
        free(buffer);
  ALOGE("SprdFm_Spk_threadentry: ok return");
  return 0;
}
```

Unisoc Confidential For longcheer

```
status_t sprdFmSpk:: fm_on()
{
        Mutex::Autolock _l(mLock);
        ALOGI("%s in",__FUNCTION__);
        if(mFmON ==true) {
                return -1;
        }
        mFmON = true;
        if((mState != STATE_START) &&(mInterfaceState == STATE_START)) {
            return    start_l(false);
        }
        ALOGI("%s out",__FUNCTION__);
        return NO_ERROR;
}
status_t sprdFmSpk:: fm_off()
{
         Mutex::Autolock _l(mLock);
         ALOGI("%s in",__FUNCTION__);
        if(mFmON ==false) {
                return -1;
        }
        mFmON = false;
        if(mState == STATE_START) {
            return    stop_l(false);
        }
        ALOGI("%s out",__FUNCTION__);
        return NO_ERROR;
}
status_t sprdFmSpk:: start()
{
        Mutex::Autolock _l(mLock);
        ALOGI("%s in",__FUNCTION__);
        mInterfaceState = STATE_START;
        return start_l(false);
}
status_t sprdFmSpk:: start_l(bool sync)
{
```

Unisoc Confidential For longcheer

```
    status_t status;
    ALOGI("%s in mState:%d,mFmON:%d",__FUNCTION__,mState,mFmON);
    if((mState ==STATE_START) ||(mFmON == false)){
        return 0;
    }
    status = sendCommand(CMD_START,sync);
    if(status == NO_ERROR) {
        mState = STATE_START;
    }
     ALOGI("%s out mState:%d,mFmON:%d",__FUNCTION__,mState,mFmON);
    return NO_ERROR;
}
status_t sprdFmSpk::stop( )
{
    Mutex::Autolock _l(mLock);
    ALOGI("%s in",__FUNCTION__);
    mInterfaceState = STATE_STOP;
    return stop_l(false);
}
status_t sprdFmSpk::stop_l(bool sync )
{
     status_t status;
    ALOGE("Sprdfm:NxpTfa_Fm_Stop in,mStart= %d",mState);
    if(STATE_START != mState) {
        ALOGE("Sprdfm: NxpTfa_Fm_Stop, not start yet.");
        return -1;
    }
    mCmdLock.lock();
    if(mAudioTrack != NULL)
        mAudioTrack->setVolume(0,0);
    mCmdLock.unlock();
    status = sendCommand(CMD_STOP,sync);
    if(status == NO_ERROR) {
        mState = STATE_STOP;
    }
    ALOGE("Sprdfm: NxpTfa_Fm_Stop out");
    return 0;
```

Unisoc Confidential For longcheer

```
}
sprdFmSpk::sprdFmSpk()
{
        mState =STATE_IDLE;
        mInterfaceState = STATE_IDLE;
        mBufferSize = 2048;
}
    sprdFmSpk::~sprdFmSpk()
{
        int ret = 0;
        ALOGE("Sprdfm: NxpTfa_Fm_Close in");
        if (ret) {
                ALOGE("Sprdfm:sem_init falied, code is");
        }
        ALOGE("Sprdfm: NxpTfa_Fm_Close out");
}
void sprdFmSpk::exit()
{
        requestExit();
        requestExitAndWait();
}
void sprdFmSpk::onFirstRef()
{
        const size_t SIZE = 256;
        char buffer[SIZE];
        snprintf(buffer, SIZE, "SprdSpkFm Thread %p", this);
        run(buffer, PRIORITY_HIGHEST);
}
void sprdFmSpk::processCmd_l()
{
        status_t status;
        while (!mCommands.isEmpty()) {
                sp<Command> event = mCommands[0];
                mCommands.removeAt(0);
                mCmdLock.unlock();
                switch (event->mCmd) {
                        case CMD_START: {
```

Unisoc Confidential For longcheer

```
                status = cmd_start_l();
            }
            break;
            case CMD_STOP: {
                cmd_stop_l();
            }
            break;
            default:
            break;
        }
        {
            Mutex::Autolock _l(event->mLock);
            if (event->mWaitStatus) {
                event->mWaitStatus = false;
                event->mCond.signal();
            }
        }
        mCmdLock.lock();
    }
}
status_t sprdFmSpk::sendCommand_l(sp<Command>& event)
{
    status_t status = NO_ERROR;
    mCommands.add(event);
     mWaitWorkCV.broadcast();
    mCmdLock.unlock();
    {
        Mutex::Autolock _l(event->mLock);
        while (event->mWaitStatus) {
            if (event->mCond.waitRelative(event->mLock, -1) != NO_ERROR) {
                event->mStatus = TIMED_OUT;
                event->mWaitStatus = false;
            }
            status = event->mStatus;
        }
    }
    return status;
```

Unisoc Confidential For longcheer

```
}
status_t sprdFmSpk::sendCommand(fm_command cmd,bool wait)
{
	Mutex::Autolock _l(mCmdLock);
	ALOGI("%s,cmd:%d",__FUNCTION__,(int)cmd);
	sp<Command> command = (Command *)new Command(cmd,wait);
	return sendCommand_l(command);
}
status_t sprdFmSpk::playback_start_l()
{
	status_t status;
	size_t minFrameCount;
	audio_stream_type_t mStreamType=AUDIO_STREAM_MUSIC;
	int sampleRate=44100;
	ALOGI("%s in",__FUNCTION__);
	status = AudioTrack::getMinFrameCount(&minFrameCount, mStreamType, sampleRate);
	if( status != NO_ERROR ) {
		ALOGE("getMinFrameCount error: status = %d\n", status);
		return	status; //
	}
	mAudioTrack = new AudioTrack(mStreamType,sampleRate , AUDIO_FORMAT_PCM_16_BIT,
AUDIO_CHANNEL_OUT_STEREO,
			(size_t)minFrameCount, AUDIO_OUTPUT_FLAG_NONE, NULL,
NULL,0,AUDIO_SESSION_ALLOCATE,android::AudioTrack::TRANSFER_DEFAULT,NULL,-1,-1,NULL);
	if(mAudioTrack == NULL){
		ALOGE("warning:set error: status = %d", status);
		status = UNKNOWN_ERROR;
	}
	status = mAudioTrack->start();
	if(status != NO_ERROR){
		ALOGE("%s:mAudioTrack->start error status:%d",__FUNCTION__,status);
	}
	return status;
}
void sprdFmSpk::playback_stop_l()
{
	ALOGI("%s in",__FUNCTION__);
	if(mAudioTrack == NULL) {
```

Unisoc Confidential For longcheer

```
            return ;
        }
        mAudioTrack-> stop();
        mAudioTrack = NULL;
}
status_t sprdFmSpk::record_start_l()
{
        status_t status;
        size_t minFrameCount;
        uint32_t frameCount;
        size_t bufCount;
         String16    opPackageName("spkfm");
         ALOGI("%s in",__FUNCTION__);
         if(!mBuffer) {
                mBuffer =(int8_t *) malloc(mBufferSize);
            if (NULL == mBuffer)
            {
                ALOGE("Sprdfm: Calloc buffer fail!");
                return NO_MEMORY;
            }
        }
        status = AudioRecord::getMinFrameCount(&minFrameCount,
                                                44100,
                                                AUDIO_FORMAT_PCM_16_BIT,

AUDIO_CHANNEL_IN_STEREO);//audio_channel_in_mask_from_count(2));//
        if (status != OK) {
            ALOGE("AudioRecord:getMinFrameCount error:%d",status);
            goto error;
        }
//确认AudioRecord callback函数不会返回超过最大值的值
        // buffer size
        frameCount = 2048 / sizeof(int16_t) / 2;
        //确认AudioRecord的总buffer size足够大
        bufCount = 2;
        while ((bufCount * frameCount) < minFrameCount) {
            bufCount+;
```

Unisoc Confidential For longcheer

```
        }
        mRecord = new AudioRecord(
                        AUDIO_SOURCE_FM_TUNER, 44100, AUDIO_FORMAT_PCM_16_BIT,
                        AUDIO_CHANNEL_IN_STEREO,
                        opPackageName,
                        bufCount
                );
        status = mRecord->initCheck();
        if (status != OK) {
                ALOGE("mRecord->initCheck error:%d",status);
                mRecord.clear();
                goto error;
        }
        mRecord->start();
        return NO_ERROR;
error:
        if(mBuffer) {
                free(mBuffer);
                mBuffer = NULL;
        }
        return status;
}
void sprdFmSpk::record_stop_l()
{
        ALOGI("%s in",__FUNCTION__);
        if(mRecord == NULL) {
                return ;
        }
        mRecord->stop();
        mRecord = NULL;
        if(mBuffer) {
                free(mBuffer);
                mBuffer = NULL;
        }
}
status_t    sprdFmSpk::cmd_start_l()
{
```

Unisoc Confidential For longcheer

```
    status_t status;
    status = record_start_l();
    if(status != NO_ERROR){
        ALOGE("record_start_l error:%d",status);
        return status;
    }
    status = playback_start_l();
    if(status != NO_ERROR) {
        record_stop_l();
    }
    return status;
}
void    sprdFmSpk::cmd_stop_l()
{
    record_stop_l();
    playback_stop_l();
}
}
```

Unisoc Confidential For longcheer